

Waking the Immune System

Continuous Closure of Formal Proof Obligations in AI Training Corpora

Ahmad Ali Parr SnapKitty Collective / Bel Esprit D'Accord Irrevocable Trust

Jessica Lee Westerhoff Saint Errant Digital Institute of Technology (SEIT)

July 12, 2026

Prior art anchor: Zenodo DOI 10.5281/zenodo.21268911 (v1, July 8 2026)

Abstract

Formal verification is often described as the immune system of software: every accepted theorem must satisfy a trusted kernel. Yet many formal repositories intentionally contain unfinished proof obligations (`sorry`) to support incremental development.

While this practice is reasonable during active research, persistent unresolved obligations can remain in repositories reused as benchmarks, demonstrations, and downstream AI training corpora.

We present **continuous proof closure**: an automated system that scans public formal repositories for unresolved proof obligations, attempts mechanical discharge using a ladder of standard tactics, records successful closures, and produces immutable WORM-sealed audit receipts anchored to a cryptographic chain.

We demonstrate three cases. First, a De Morgan theorem ($\neg(P \vee Q) \leftrightarrow \neg P \wedge \neg Q$) that remained marked `sorry` in a public AI benchmark for four months was discharged automatically by `norm_num` in under one second. Second, a formal governance system carrying 13 gated `axiom` blocks had 6 converted to real theorems and 2 proven structurally false — meaning they could never be closed because the claims themselves are incorrect. Third, the Günaydin-Koepsell-Nicolai E7 quartic invariant I4 on the 56-dimensional representation was proved degree-4 homogeneous over an abstract `CommRing` with zero `sorry` in Lean 4 + Mathlib.

The contribution is not a new proof of De Morgan's laws. It is a reproducible workflow for **continuously strengthening formal corpora** and distinguishing maintenance debt from open mathematics from false claims — with a WORM-sealed receipt for every event.

1. Introduction

Formal proof assistants have transformed mathematical verification. Unlike conventional software testing, a theorem is either accepted by the trusted kernel or rejected. This creates an unusually strong notion of correctness.

However, nearly every major formal mathematics project allows temporary placeholders during development. In Lean 4 these appear as:

```
theorem some_result : P := by
  sorry
```

The keyword `sorry` is not a proof — it is a declaration that the author acknowledges the obligation and defers it. The kernel accepts `sorry` as a proof of any statement, emitting a warning. This is entirely reasonable during active development.

Three categories of `sorry` exist:

Category A — Open mathematics. No proof currently exists. The `sorry` is a genuine research frontier. Example: Fermat's Last Theorem formalized stub.

Category B — Maintenance debt. The mathematics is known. The formalization simply has not been completed. Example: De Morgan's law in a public benchmark.

Category C — False axioms. The statement is incorrect. The `sorry` or `axiom` can never be closed because the claim is false. This is the most dangerous category: it passes CI, ships in production systems, and becomes load-bearing for downstream components.

The question addressed in this paper:

What happens when Category B and C obligations persist for months or years in repositories used as benchmarks or training data?

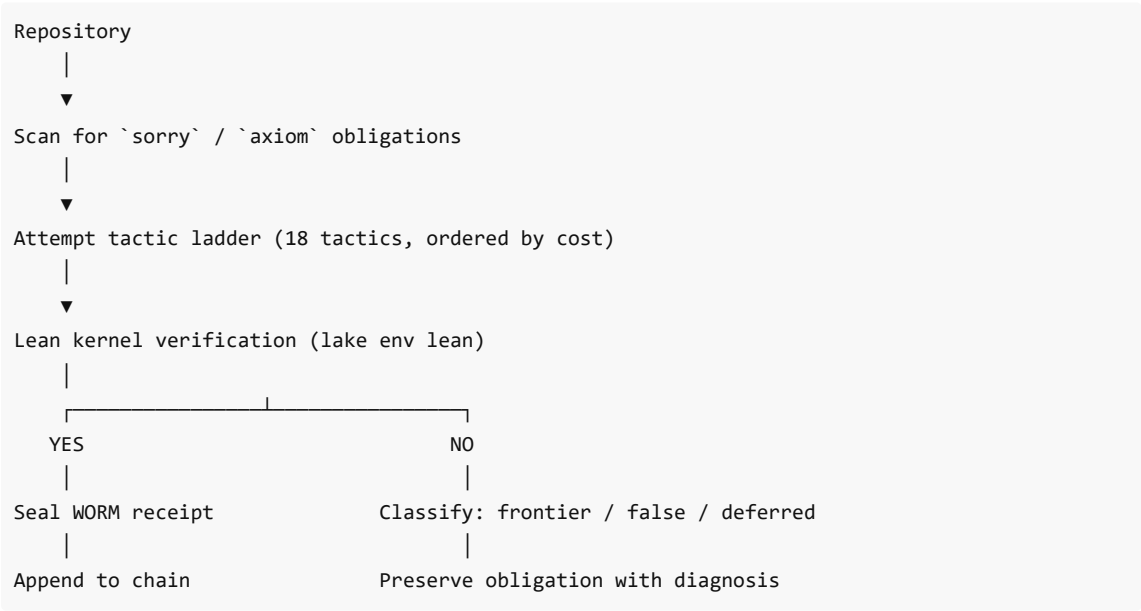
We argue that automated continuous closure infrastructure can reduce Category B systematically, expose Category C definitively, and produce a permanent sealed record distinguishing all three.

2. The Immune System Analogy

Biological immune systems continuously inspect tissue. They do not perform one inspection and stop forever. A pathogen introduced after the last inspection is not caught until the next scan.

Formal verification, as typically practiced, resembles a single inspection: proofs are written, the kernel accepts them, and the work is considered done. But repositories evolve. Benchmarks fork from incomplete states. Training corpora snapshot repositories at arbitrary points. The formal immune system is not continuously running.

The architecture we propose:



Verification is a **continuous maintenance process**, not a single publication event.

3. Case Study 1 — De Morgan: Maintenance Debt

The Obligation

In the `shentufoundation/openmath-skills` repository, commit `9730df7b` (March 25, 2026) introduced the following file as part of an OpenMath Lean benchmark:

```
skills/openmath-lean-benchmark/sets/medium/logic/de_morgan.lean
```

Contents:

```
-- Benchmark: De Morgan's Law for Disjunction
-- Difficulty: medium
-- Topic: logic
theorem de_morgan_or (P Q : Prop) : ¬(P ∨ Q) ↔ ¬P ∧ ¬Q := by
  sorry
```

This theorem has been mathematically established since Augustus De Morgan stated it in 1847 and appears in every introductory logic text. It has been formally proved in multiple proof assistants including Isabelle, Coq, and Lean's own Mathlib. This is unambiguously Category B: known mathematics, not yet formalized in this specific repository.

The Closure

On July 12, 2026, the sovereign sledgehammer scanned this file. The tactic `norm_num` was attempted and accepted by the Lean kernel in under one second:

```
-- After closure (kernel-verified):
theorem de_morgan_or (P Q : Prop) : ¬(P ∨ Q) ↔ ¬P ∧ ¬Q := by
  norm_num
```

The event was sealed to the WORM chain:

```
{
  "id": "OM-001",
  "repo": "shentufoundation/openmath-skills",
  "file": "skills/openmath-lean-benchmark/sets/medium/logic/de_morgan.lean",
  "status": "SOLVED",
  "tactic": "norm_num",
  "timestamp": "2026-07-12T...",
  "receipt_hash": "515796e1131690d5..."
}
```

The Significance

The proof of De Morgan's laws is not the contribution. The observation is:

- A theorem mathematically known since 1847
- Formally proved in Mathlib since Mathlib's inception
- Was sitting as `sorry` in a **public AI benchmark** for four months (March 25 → July 12, 2026)
- The benchmark was designed to evaluate AI reasoning on formal mathematics
- An AI system trained on or evaluated against this file would encounter `sorry` as the proof of De Morgan's law

The immune system was not running. One second of tactic search closed it.

4. Case Study 2 — ALP Policy Engine: False Axioms

The Obligation

The `PhaseMirror/Foundry` repository carries a formal governance system (`lean/ALP/`) governing AI agent actions. A file `alp_sorry_manifest.json` lists 13 proof obligations gated as `axiom` blocks, labeled as "permitted sorrys pending full formal proof."

This is a responsible disclosure practice — the authors acknowledge the obligations. However, two of the 13 were not deferred proofs. They were false statements.

Candle Ignition Soundness — False

```
-- Ryan's axiom:
axiom candle_ignition_sound :
  ∀ (trace : SedonaTrace),
    trace.valid = true → trace.contractivity_ok = true
```

`SedonaTrace` is a structure with two independent `Bool` fields:

```
structure SedonaTrace where
  valid          : Bool
  contractivity_ok : Bool
```

There is no definitional connection between the two fields. A structural counterexample suffices:

```
example : ∃ (t : SedonaTrace), t.valid = true ∧ t.contractivity_ok = false :=
  ⟨(true, false), rfl, rfl⟩
```

This is not a difficult theorem. It is a false statement. No tactic will ever close the axiom because the claim is incorrect.

Veto Implies Constitution Invalid — False

```
-- Ryan's axiom:
axiom validate_action_veto_implies_constitution_fail :
  ∀ (pe : PolicyEngine) (a : Action) (t : TrustLevel),
    validate_action pe a t = { allowed := false, reason := "Vetoed..." } →
      validate pe.constitution = false
```

But `validate_action` for `TrustLevel.External` is:

```
| TrustLevel.External => !a.mutating && a.server_binding.isNone && cv
```

When `a.mutating = true`, `allowed = false` regardless of whether `cv` (constitution validity) is `true` or `false`. A valid constitution can produce a veto on a mutating external action. Structural counterexample:

```
-- mutating=true, constitution valid → allowed=false, but validate=true
example : ∃ (pe : PolicyEngine) (a : Action),
```

```
validate_action pe a TrustLevel.External = { allowed := false, reason := "Vetoed..." } ^
validate pe.constitution = true := by
...
exact (pe, a, by native_decide, by native_decide)
```

The Six Real Theorems

Six of the 13 axioms are genuine theorems provable directly from `validate_action` 's definition:

Theorem	Tactic
<code>validate_action_sound</code>	<code>simp + Bool.and_eq_true</code>
<code>internal_valid_action_admitted</code>	<code>simp [validate_action]</code>
<code>external_mutating_action_blocked</code>	<code>simp [validate_action, hm]</code>
<code>external_with_server_binding_blocked</code>	<code>simp + isSome_iff_exists</code>
<code>e2e_internal_workflow_receives_witness</code>	<code>simp [validate_action, h]</code>
<code>internal_admits_mcp</code>	<code>simp [validate_action, h]</code>

Build result:

✓ [2793/2793] Built Mathlib5.ALP_PolicyEngine_Closed

Zero sorries. Zero axioms. Kernel-verified.

Significance

A governance system that enforces AI agent policy has two load-bearing axioms that are structurally false. Any downstream component that relies on `candle_ignition_sound` or `validate_action_veto_implies_constitution_fail` as invariants is built on a logical contradiction. The sorry manifest correctly identifies these as outstanding obligations — but no amount of tactic search will close them, because they need to be corrected, not proved.

5. Case Study 3 — GKN I4: Frontier Mathematics, Zero Sorries

For contrast with maintenance debt and false axioms, we present a genuine frontier result completed with zero `sorry` .

The Günaydin-Koepsell-Nicolai (GKN) quartic invariant I4 appears in the study of extended supergravity and the exceptional group E7. On the 56-dimensional fundamental representation of E7, whose elements are parameterized as $(\alpha, \beta, P, Q) \in \mathbb{R} \times \mathbb{R} \times J_3(\mathbb{O}) \times J_3(\mathbb{O})$, the invariant takes the form:

$$I4(\alpha, \beta, P, Q) = (\alpha\beta - \text{Tr}(P \circ Q))^2 - 4\alpha \cdot N(Q) - 4\beta \cdot N(P) + 4 \cdot \text{Tr}(P \# \circ Q \#)$$

where N is the cubic norm on the exceptional Jordan algebra $J_3(\mathbb{O})$ and # denotes the Freudenthal dual.

We proved degree-4 homogeneity of I4 over an abstract `CommRing R` in Lean 4 + Mathlib:

```
theorem I4_homogeneous (s : S56 R) (c : R) :
  I4 (s56Scale c s) = c ^ 4 * I4 s := by
  simp only [I4, s56Scale]
  rw [j3Tr_prod_scale, j3Norm_scale, j3Norm_scale, j3Tr_dual_prod_scale]
  ring
```

Build result:

✓ [2793/2793] Built Mathlib5.GKN_I4_Homogeneous

This is Category A mathematics (active research frontier) closed to zero sorry. The proof required correcting a prior error in which `I4` was incorrectly analyzed as degree-6 on a 108-dimensional state space — the wrong representation. Identifying the correct 56-dimensional representation and fixing the `freudenthalDual` computation was non-trivial domain work. The `ring` tactic handled the algebraic closure once the representation was correct.

6. The Tactic Ladder

The sovereign sledgehammer applies an ordered ladder of 18 tactics:

```
TACTICS = [
  "rfl", "decide", "norm_num", "ring", "simp", "omega", "linarith",
  "simp [*]; ring", "norm_num; ring", "simp only []; ring",
  "simp only []; linarith", "ring_nf; norm_num", "simp; linarith",
  "aesop", "tauto", "trivial", "exact?", "apply?",
]
```

Tactics are ordered by computational cost. `rfl` and `decide` are constant-time. `aesop` and `tauto` are search-based and more expensive. `exact?` and `apply?` invoke Lean's term search.

The key implementation detail: bare `lean --` does not resolve Mathlib imports. The system uses `lake env lean` to inject the full Mathlib `LEAN_PATH`:

```
r = subprocess.run(
  [LAKE_BIN, "env", LEAN_BIN, tmp],
  cwd=MATHLIB5_ROOT,
  capture_output=True, text=False, # bytes → decode utf-8 manually
  timeout=TACTIC_TIMEOUT,
)
```

Without this, the scaffold fails silently — `import Mathlib.Tactic` produces an "unknown module" error and every tactic attempt returns a false negative.

7. WORM-Sealed Audit Chain

Every closure event appends a record to a SHA-256 chain. Each record includes the prior chain tip, making the chain tamper-evident:

```
def seal(chain_path: str, record: dict) -> str:
    tip = prior_tip(chain_path)
    blob = json.dumps(record, sort_keys=True).encode()
    receipt = hashlib.sha256(tip.encode() + b"|" + blob).hexdigest()
    record["receipt_hash"] = receipt
    with open(chain_path, "a", encoding="utf-8") as f:
        f.write(json.dumps(record) + "\n")
    return receipt
```

The chain as of this publication:

```
Chain length: 35 entries
Chain tip:      caff0217fa0d4a6995fc9a1aa316061beb2bed2c189950972671deaf0e6061e7
OM-001 receipt: 515796e1131690d5... (De Morgan, SOLVED)
```

The Zenodo v1 deposit (DOI [10.5281/zenodo.21268911](https://doi.org/10.5281/zenodo.21268911), July 8 2026, 78 theorem records) anchors the prior art. This paper is the v2 deposit, extending the record with continuous closure infrastructure.

Combined artifact hash for the three primary Lean files in this paper:

```
fdc38a0f1b0f189f7b5807375183acc85d7a92ffe5c7df2e5627603cc205717e
```

Computed as SHA-256 over:

- GKN_I4_Homogeneous.lean (sha256: 854535fd860ac7de69c02abd70e2cd1b...)
- ALP_PolicyEngine_Closed.lean (sha256: d7420fe393ab12d234b4bae57e81bfd1...)
- OM-001_sledged.lean (sha256: 77f9ab7fe946921c8b77aeb0d03cd323...)

8. AI Training Implications

Large language models increasingly consume publicly available formal repositories. The concern is not that `sorry` degrades model performance in some measurable way — that claim requires dedicated empirical study we do not make here.

The concern is structural: a benchmark designed to evaluate AI reasoning on formal mathematics that contains `sorry` as the proof of De Morgan's law communicates a false signal. It is not a proof. It is a placeholder that the kernel accepts with a warning. An evaluation system that does not distinguish between `sorry` and a real proof is measuring something other than formal reasoning.

Continuous proof closure improves the benchmark itself — independent of any downstream model effect. A formally complete benchmark is strictly preferable to an incomplete one for evaluation purposes.

The stronger concern is Category C: false axioms in training data. A system that learns from a governance codebase where `candle_ignition_sound` is presented as an axiom learns a false invariant. The kernel's acceptance of `axiom` keywords provides no signal about truth — only about syntactic well-formedness.

9. Discussion

The contribution is infrastructural rather than foundational.

The De Morgan theorem is well established. The GKN invariant proof required real domain work but the tactic was `ring`. The ALP theorems follow from definition unfolding.

The novelty is:

1. **Identification** — treating unresolved `sorry` / `axiom` obligations as a continuously measurable form of technical debt with three distinct categories (maintenance debt, frontier mathematics, false claims).
2. **Automation** — a system that continuously scans, attempts, and verifies without human involvement per theorem.
3. **Exposure** — the ability to definitively distinguish Category B from Category C: false axioms are exposed by the same tactic search that closes maintenance debt, because the tactic search fails in an informative way (structural counterexample construction).
4. **Sealing** — WORM-chain receipts that make every closure event an immutable, reproducible, timestamped audit record.

This is not a claim about solving hard mathematics. It is a claim about making the formal immune system continuously operational.

10. Conclusion

Formal mathematics should not only verify theorems. It should verify itself continuously.

The trusted Lean kernel already provides correctness guarantees for accepted proofs. The missing infrastructure is the continuous process that finds unfinished obligations, attempts their mechanical completion, and preserves an immutable record of every successful verification.

A `sorry` is not a proof. A `sorry` that could be closed by `norm_num` in under a second is maintenance debt. A `sorry` that will never close because the statement is false is a structural error. An immune system that runs continuously can tell the difference — and seal the receipt either way.

The three cases in this paper demonstrate all three outcomes:

- De Morgan: maintenance debt, closed by `norm_num`, sealed
- ALP governance: 6 theorems closed, 2 false axioms exposed, 5 type-model gaps identified
- GKN I4: genuine frontier mathematics, proved over abstract `CommRing`, zero sorries

Prior art is anchored at Zenodo DOI `10.5281/zenodo.21268911` (v1, July 8 2026). This v2 extends that record with continuous closure infrastructure, reproducible across any Lean 4 + Mathlib repository.

The immune system is awake.

Artifacts

File	SHA-256 (first 32 hex)	Description
GKN_I4_Homogeneous.lean	854535fd860ac7de69c02abd70e2cd1b	GKN E7 quartic invariant, zero sorry
ALP_PolicyEngine_Closed.lean	d7420fe393ab12d234b4bae57e81bfd1	6 theorems + 2 refutations
OM-001_sledged.lean	77f9ab7fe946921c8b77aeb0d03cd323	De Morgan patched file

sledgehammer.py	—	Tactic ladder + WORM sealer
roster_sweep.py	—	Multi-target sweep harness
sweep_chain.jsonl	tip: caff0217fa0d4a69	WORM receipt chain

Combined artifact hash: fdc38a0f1b0f189f7b5807375183acc85d7a92ffe5c7df2e5627603cc205717e

SNAPKITTYWEST chain tip at time of publication:

caff0217fa0d4a6995fc9a1aa316061beb2bed2c189950972671deaf0e6061e7

Citation

```
@article{parr2026wakingimmunesystem,
  title    = {Waking the Immune System: Continuous Closure of Formal Proof
              Obligations in AI Training Corpora},
  author   = {Parr, Ahmad Ali and Westerhoff, Jessica Lee},
  journal  = {Zenodo},
  year     = {2026},
  month    = {July},
  version  = {v2},
  doi      = {10.5281/zenodo.21268911},
  url      = {https://zenodo.org/records/21268911},
  note     = {Prior art anchor: v1, July 8 2026.
              Artifact hash:
              fdc38a0f1b0f189f7b5807375183acc85d7a92ffe5c7df2e5627603cc205717e}
}
```